

System Architecture — Crowdsourced Bus Tracking & Complaint App

1. Overview

An AI-chatbot-first transit app for public buses, built around four data sources: crowdsourced passenger GPS, QR-based bus identity check-ins, peer-to-peer route condition reports, and structured complaints. The system fuses these into a live, trustworthy picture of where buses are, how full they are, and what's wrong with them — without requiring any active hardware installed on the bus itself.

Core problem it solves: informal/under-regulated bus networks (like in Karachi) have no official real-time tracking, no complaint channel that reaches the right authority, and no way for riders to know which bus on a route is actually the better choice right now.

Product stage: real, long-term deployable product (not a one-off hackathon demo).

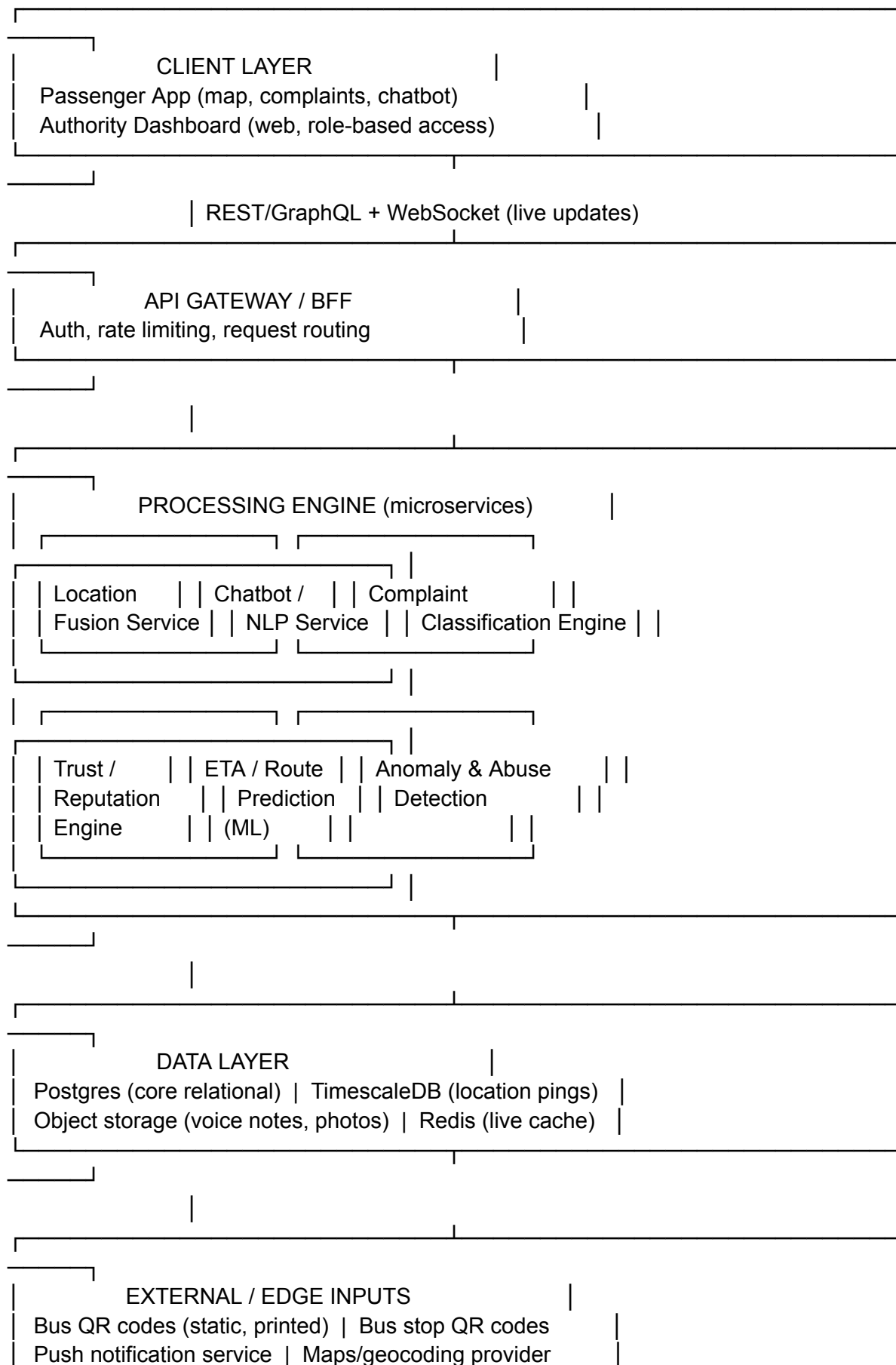
Hardware scope (MVP decision): QR code only, no active GPS/BLE tracker device. This keeps deployment cost near-zero and avoids needing bus-operator buy-in for hardware installation — but it means live tracking depends on having at least one rider with the app open on each bus (the "cold-start problem," addressed in Section 6).

Primary interface: standalone mobile app (no WhatsApp integration in this phase).

2. Design Principles

1. **No single point of failure in data** — never trust one phone's GPS alone; always fuse multiple signals.
 2. **Incentivize participation** — the system only works if people check in and report, so trust/reward mechanics are core, not an add-on.
 3. **Degrade gracefully** — when live data is thin, fall back to historical patterns instead of showing nothing.
 4. **Privacy by default** — raw location data is short-lived and aggregated; users are never individually trackable by other users.
 5. **Authority-ready** — complaint and illegal-bus data must be exportable/actionable for transport authorities, not just visible in-app.
-

3. High-Level Architecture



4. Data Ingestion Layer

Source	What it captures	Trigger
QR check-in (on bus)	Bus ID, route, timestamp	Passenger scans on boarding
QR check-in (at stop)	"Bus not here yet" / wait-time signal	Passenger scans while waiting
Background GPS	Live position, tagged to checked-in bus ID	Continuous while checked in, opt-in
Complaint form	Structured issue (overspeeding, AC broken, illegal bus) + optional photo/voice	Manual submission
Chatbot conversation	Free-form text or voice note, parsed into intent + entities	User-initiated
Peer condition reports	Short structured alert ("traffic here", "bus skipped stop")	Manual, lightweight tap-to-report

All location data is tagged with a **session ID**, not a persistent user identity, before it leaves the device — this is what makes later anonymization possible without redesigning the pipeline.

5. Processing Engine

5.1 Location Fusion Service

- Ingests all GPS pings tagged to a given bus ID within a rolling time window.
- Weights each ping by the sending user's current trust score (Section 8) and signal quality (GPS accuracy radius reported by the OS).
- Outputs a single fused position + confidence interval per bus, published to Redis for low-latency reads and pushed via WebSocket to riders following that route.
- If zero live pings exist for a bus, falls back to last-known-position + scheduled/historical timing pattern, clearly labeled as "estimated, not live" in the UI.

5.2 Chatbot / NLP Service

- Handles Urdu, English, and Roman Urdu, including code-switched input (e.g. "yeh bus late hai kya").
- Speech-to-text for voice notes, tuned for code-switched Pakistani speech patterns (a fine-tuned/adapted STT model is likely needed — generic English STT will fail on Roman Urdu phonetics).
- Intent classifier routes to one of: file a complaint, ask for live bus location, ask which bus is better for a route, general query.
- On "file a complaint" intent, extracts entities (bus number, issue type) where possible and sends the user directly to the pre-filled complaint form/link rather than trying to complete the whole flow conversationally.

5.3 Complaint Classification Engine

- Categorizes incoming complaints (overspeeding, AC/mechanical, illegal/unregistered bus, driver behavior, other).
- Deduplicates near-identical complaints about the same bus/route/time window into a single case with multiple reporters (this is what turns individual complaints into credible evidence).
- Routes case to a human moderation queue before anything is forwarded to the authority dashboard.

5.4 Trust / Reputation Engine

See Section 8 — this is the anti-spam backbone of the whole system.

5.5 ETA / Route Prediction (ML)

- Trained on historical fused-location data plus schedule metadata.
- Predicts arrival time per stop and, for "which bus is better" queries, compares live-estimated arrival + crowd-density across all buses currently serving that route.

5.6 Anomaly & Abuse Detection

- Flags physically impossible GPS jumps (spoofing).
- Flags accounts with unusually high report volume in short windows (spam pattern).
- Cross-references phone accelerometer data against reported "overspeeding" complaints as a secondary validation signal, not the sole trigger.

6. Solving the Cold-Start Problem (QR-only constraint)

Because there's no active hardware, a bus with zero checked-in riders is invisible.

Mitigations:

1. **Incentivized check-in** — trust points for scanning, redeemable for perks (priority complaint handling, badges, visible reputation).

2. **Stop-side QR** — even without boarding, a rider waiting at a stop can report "no bus yet," which is itself useful data and keeps engagement up between rides.
 3. **Graceful degradation** — always show *something* (scheduled time / last-known position with a clear "not live" label) instead of a blank screen.
 4. **Crowd-density as a side benefit** — check-in/check-out counts on a bus double as a rough occupancy estimate ("this bus is empty, next one is full") — a genuinely useful feature that falls out of the QR-only design for free.
-

7. Complaint Workflow (end to end)

1. User submits complaint (via form or chatbot) → tagged with bus ID (from QR history or manual entry), issue type, optional media.
 2. Complaint Classification Engine deduplicates against existing open cases for that bus/window.
 3. Case enters moderation queue (human review) to filter spam/malicious reports.
 4. Approved case appears on the Authority Dashboard, grouped by bus/route, with reporter count as a credibility signal.
 5. Repeated illegal-bus reports across multiple independent sessions auto-flag the case as high-priority.
-

8. Trust / Reputation Engine (detail)

- Every session/user starts at a neutral baseline score.
 - Score increases: consistent QR check-ins that match other riders' fused location, complaints that get corroborated by other reporters, no history of flagged anomalies.
 - Score decreases: GPS spoofing detected, complaints repeatedly rejected in moderation, unusually high-frequency reporting.
 - Score affects: weight in location fusion, whether a complaint skips extra review, visibility of the user's reports to others.
 - This is what prevents the entire crowdsourced model from being gamed — it has to be designed in from day one, not bolted on later.
-

9. Data Layer

Store	Used for	Why
PostgreSQL	Users, buses, routes, complaints, cases	Relational integrity, transactional
TimescaleDB (or similar time-series DB)	Raw and fused location pings	High write volume, time-windowed queries

Redis	Latest fused bus positions, session cache	Low-latency reads for live map
Object storage (S3-compatible)	Voice notes, complaint photos	Cheap, scalable blob storage

Retention policy: raw location pings retained only long enough for fusion + short-term ETA training (e.g. 30–90 days), then aggregated/anonymized or discarded. Complaint case data retained longer since it may be needed by authorities.

10. Client Layer

Passenger app

- Live map with fused bus positions (confidence-labeled), route search, "which bus is better for my route" comparison
- Complaint form (pre-fillable from chatbot)
- Chatbot (text + voice), multilingual
- Personal trust score / reporting history

Authority dashboard (web, role-based)

- Complaint case queue, grouped and prioritized
 - Illegal-bus cluster flags
 - Route performance and overspeeding heatmaps
 - Export/reporting tools
-

11. Security & Privacy

- Session-based location tagging, not persistent user tracking, at the point of collection.
 - Raw pings never exposed to other end users — only fused, aggregated bus positions are.
 - Role-based access control on the authority dashboard.
 - Rate limiting and anomaly detection at the API gateway to blunt spam/abuse before it reaches the processing layer.
 - Clear data retention limits (Section 9).
-

12. Suggested Tech Stack

- **Mobile app:** Flutter or React Native (cross-platform, one codebase for Android/iOS)

- **Backend services:** Node.js or Python (FastAPI) microservices
 - **Real-time updates:** WebSocket (Socket.io) or MQTT for live position push
 - **Databases:** PostgreSQL, TimescaleDB, Redis, S3-compatible object storage
 - **NLP/Chatbot:** LLM API (e.g. Claude) for intent parsing + a speech-to-text model with Urdu/Roman Urdu support, fine-tuned if generic accuracy is too low
 - **ML (ETA, anomaly detection):** Python (scikit-learn/PyTorch), trained offline, served via a lightweight inference API
 - **Maps:** Google Maps SDK or Mapbox
 - **Infrastructure:** containerized services (Docker) behind an API gateway, deployable on any major cloud
-

13. Roadmap

Phase 1 — MVP: QR check-in, crowdsourced GPS fusion (basic), complaint form, single-language chatbot, passenger app only.

Phase 2 — V1: Trust/reputation engine, multilingual + voice chatbot, authority dashboard, ETA prediction.

Phase 3 — V2: Crowd-density estimation, anomaly/abuse detection, route comparison ("which bus is better"), stop-side QR reporting.

Phase 4 — Scale: Explore active hardware (BLE/GPS trackers) on partnered routes once there's traction and authority buy-in, without breaking the QR-based fallback for everyone else.

14. Unique Differentiators (summary)

1. Trust/reputation-weighted crowdsourced fusion instead of trusting any single GPS source.
2. QR check-in doubles as both identity anchor and free crowd-density signal — no extra hardware needed.
3. Complaint deduplication and corroboration turns scattered reports into credible evidence for authorities.
4. Graceful degradation — never a blank screen, always a best-effort estimate.
5. Phone-sensor cross-validation for overspeeding complaints, not just self-reported text.